thedesignengineer

Where NOT to get Traped : PART I

TOP 5 ANGULAR CODING CHALLENGES THAT ELIMINATE 80% OF CANDIDATES

(Experienced | Senior | Lead Level)
With real interview traps + clean code

kitturkoustubh@gmail.com | IG @thedesignenginner.in

thedesignengineer

Why These 5 Matter

These are not “hard” problems.
They are filter problems.

Interviewers use them to check:

- Async thinking
- Architecture discipline
- RxJS maturity
- Performance awareness
- Production mindset

Fail these → rejection.
Explain these calmly → cleared.

kitturkoustubh@gmail.com | IG @thedesignenginner.in

thedesignengineer

Debounced Search
(RxJS Classic Trap)

Problem : Build a search input

- Debounce user input
- Cancel previous API calls
- Show only latest results

❌ ***Common Trap***

Using mergeMap → race conditions

⭐ **Difficulty:** ⭐⭐⭐⭐⭐

🎯 ***Interviewer Signal:***

Knows real async UX problems.

kitturkoustubh@gmail.com | IG @thedesignenginner.in

thedesignengineer

Debounced Search
(RxJS Classic Trap)

✅ ***Correct Solution***

```
search$ = this.searchControl.valueChanges.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap((term) => this.api.search(term).pipe(catchError(() => of([]))))
)
```

🧠 ***Why switchMap?***

Cancels previous requests automatically.

kitturkoustubh@gmail.com | IG @thedesignenginner.in

API Call with Loader + Error Handling

Problem : Call API and manage

- Loader
- Success
- Error
- Retry (optional)

❌ ***Common Trap***

- Loader stuck forever
- Error not handled
- UI logic in component

⭐ **Difficulty:** ⭐⭐⭐⭐

🎯 ***Interviewer Signal:***
Production-ready mindset.

kitturkoustubh@gmail.com | IG @thedesignenginner.in

thedesignengineer

API Call with Loader + Error Handling

✅ ***Correct Solution :*** *Clean Service-Based Pattern*

```
getUsers() {
  return this.http.get<User[]>('/api/users').pipe(
    retry(2),
    catchError(err ⇒ {
      this.error$.next(true);
      return throwError(() ⇒ err);
    })
  );
}

this.loading = true;
this.service.getUsers().subscribe({
  next: data ⇒ this.users = data,
  error: () ⇒ {},
  complete: () ⇒ this.loading = false
});
```

kitturkoustubh@gmail.com | IG @thedesignenginner.in

Reactive Form with Async Validation

Problem : Validate username/email from server

- without blocking UI

❌ ***Common Trap***

- Calling API inside component manually

⭐ **Difficulty:** ⭐⭐⭐⭐

🎯 ***Interviewer Signal*** / 🧠 ***Analogy***

Background verification, not a roadblock.

kitturkoustubh@gmail.com | IG @thedesignenginner.in

thedesignengineer

Reactive Form with Async Validation

✅ ***Correct Solution :*** *Async Validator*

```
usernameValidator(): AsyncValidatorFn {
  return (control: AbstractControl) ⇒
    this.api.checkUsername(control.value).pipe(
      map(isTaken ⇒ isTaken ? { taken: true } : null),
      catchError(() ⇒ of(null))
    );
}


this.form = this.fb.group({
  username: ['', null, this.usernameValidator()]
});
```

kitturkoustubh@gmail.com | IG @thedesignenginner.in

Change Detection + Performance Fix

Problem : Large list rendering slowly

❌ ***Common Trap***

- Blaming Angular instead of change detection

⭐ **Difficulty:** ⭐⭐⭐⭐⭐

🎯 ***Interviewer Signal*** / 🧠 ***Why it works***

- OnPush limits checks
- trackBy prevents re-rendering

🎯 **Goldman / Banking favorite**

kitturkoustubh@gmail.com | IG @thedesignenginner.in

Change Detection + Performance Fix

✅ ***Correct Solution :***

```
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class ListComponent {
  trackById(index: number, item: Item) {
    return item.id;
  }
}
```

```
<li *ngFor="let item of items; trackBy: trackById">
```

kitturkoustubh@gmail.com | IG @thedesignenginner.in

thedesignengineer

Memory Leak Prevention (Silent Killer)

Problem : Subscriptions never cleaned → leaks

❌ ***Common Trap***

- Manual unsubscribe everywhere

★ **Difficulty:** ★★★★★

🎯 ***Interviewer Signal***

- Knows Angular lifecycle deeply.

✅ ***Even Better***

- Use async pipe wherever possible.

kitturkoustubh@gmail.com | IG @thedesignenginner.in

thedesignengineer

Memory Leak Prevention (Silent Killer)

✅ ***Correct Solution :*** Best Practice

```
private destroy$ = new Subject<void>();

ngOnInit() {
  this.stream$
    .pipe(takeUntil(this.destroy$))
    .subscribe();
}

ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}
```

kitturkoustubh@gmail.com | IG @thedesignenginner.in

Common Traps Interviewers Expect

- mergeMap for search ❌
- API calls in components ❌
- Global state for local UI ❌
- No error state ❌
- No cleanup logic ❌

These are rejection triggers.

kitturkoustubh@gmail.com | IG @thedesignenginner.in

What Interviewers Are REALLY Testing

They're asking:

- Can this person handle real production Angular apps under async pressure and team scale?

Not :

- Syntax
- Fancy operators
- Framework trivia

kitturkoustubh@gmail.com | IG @thedesignenginner.in

thedesignengineer

Mic-Drop Line (Memorize This)

“I focus on predictable data flow, controlled change detection, and clean async boundaries — not just making screens work.”

Follow for UI + UX + AI + Engineering ***Brainstorming***..

Comment “PART 2” if you want more coding examples.

kitturkoustubh@gmail.com | IG @thedesignenginner.in

Save this for future reference